SWP-PYTHON 5AHWII

WS+SS 2026

Sabo RUBNER - RubnS - sabo.rubner@htlinn.ac.at

20. Oktober 2026

Übersicht

SWP 5A - Python

- Test-Termine
- Rahmenbedingungen
- Stoff-Übersicht

Test-Termine

- siehe WebUntis
- Semesterende 2026.12.23 !!!

Github Folien Link

- https://github.com/htlinnrain/python

Git Repos G1

- Böckle https://github.com/Mo318-Mo/swp_python_boeckle
- Bodner https://github.com/lisabodner/swp-projekte-bodner-5ahw.git
- Brunnerhttps://github.com/TomBrnr/SWP_Python_Brunner.git
- Dulic https://github.com/dendiedas/Dulic_Python
- Falkensteiner https://github.com/PhilippFalkensteiner/SWP5_Python_Falkensteiner.git
- Geisler https://github.com/eligeisler/SWP_Python
- Gundolf https://github.com/MZmaxi6212/SWP-Phyton-Gundolf
- Kirchner https://github.com/Benjjiii00/SWP-Python-5ahw-Kirchner
- Klingenschmidhttps://github.com/Tintenkiller1207373/Klingenschmid_SWP_Python
- Liepert https://github.com/Tob-123/Liepert_SWP_Rubner

Git Repos G2

- Niederwieser `https://github.com/niwiflo/SWP_Python_Niederwieser`
- Oberladstätter `https://github.com/mathiasoberladi-spec/SWP-Python-Mathias`
- Oberlechner `https://github.com/voberl/SWP_Python_Oberlechner`
- Pacher `https://github.com/jupacher/Pacher_Swp_Python_git`
- Roncov `https://github.com/JustAGuy67/2026-SWP-Python-Roncov`
- Schattel `https://github.com/Fritz-derEchte/swp_rub/tree/main`
- Scheyrer `https://github.com/leonie-scheyrer/RubnS_SWP`
- Sieberer `https://github.com/PaulSieberer/SWP_Python_Sieberer`
- Viertl `https://github.com/Simi-cpu/2026-SWP-PYTHON_Viertl`
- Zeindl `https://github.com/jonas037321/SWP_Python_Zeindl`

Rahmenbedingungen

- drei oder vier Tests
- mehr als die Hälfte der Einheiten gefehlt - P5 Prüfung
- zu spät kommen - Tafel
- Mitarbeit: jeder startet bei Befriedigend, plus -0.25 und minus +0.5
- Mitarbeit 30% Test 70% Gesamtnote - 1HJ 45% 2HJ 55%
- jeder Test muss positiv sein, wenn nicht P5 Prüfung
- Teststoff siehe Fragensammlung
- HUs gehören in ein eigenes GIT Repo

Stoff-Übersicht

- Python Theorie und einzelne Aufgaben
- verkettete Listen mit Python
- 2tes HJ individuelles Projekt in Python

Ablauf KW39

- Einführung
- Folien via GIT
- Git Repo für 2026 SWP-PYTHON anlegen
- Python downloaden, HalloWelt programmieren

Ablauf KW38

- **Wer den Code nicht versteht, steuert nicht – er fliegt blind**
  - **Debugging & Verifikation:** KI generiert schnell funktionierenden Code, baut aber regelmäßig subtile Logikfehler, Race Conditions oder Sicherheitslücken ein. Wer Grundlagen, Datenstrukturen und Systemverhalten nicht beherrscht, kann KI-Code weder prüfen noch reparieren.
  - **Die Pilot-Analogie:** Ein moderner Jet fliegt meist auf Autopilot. Trotzdem muss der Pilot das Flugzeug im Detail verstehen, um bei Systemausfällen oder Turbulenzen die Kontrolle zu übernehmen.
- **Programmieren ist Problemübersetzung, nicht Syntax-Tippen**
  - **Architektur & Kontext:** Das Generieren von Syntax macht nur einen Bruchteil der Softwareentwicklung aus. Die eigentliche Ingenieursleistung liegt im Systementwurf, dem Schnittstellendesign, der Leistungsskalierung und dem Verstehen der realen Problemstellung.
  - **Präzise Anforderung:** Um einer KI überhaupt den richtigen Auftrag (Prompt) zu geben, muss das Problem auf technischer Ebene bereits durchdrungen sein. Vage Eingaben führen zu unbrauchbarem Code.

Ablauf KW38

- **Haftung und der Multiplikator-Effekt**
  - **Verantwortung:** Bei Fehlern in kritischen Systemen (Medizintechnik, Steuerungs- und Regelungstechnik, Automobil oder Finanzen) haftet kein Sprachmodell, sondern der Entwickler mit seiner Unterschrift.
  - **Neue Rollenverteilung:** KI eliminiert rein mechanische Tipparbeit („Code Monkeys"). Entwickler, die Informatikgrundlagen beherrschen und KI als Werkzeug einsetzen, werden nicht arbeitslos, sondern um ein Vielfaches produktiver.

Ablauf KW39

- Python2/Python3 (in P3 mit Abwärtskompabilität gebrochen)
- iterativ und rekursiv besprechen
- Unterschied Prozesse Threads
- keine Datentypen (type) dynamische/statische Typisierung
- aber type hints
- funktionale und objektorientierte Programmierung möglich
- NichtInterpreter und Interpreter Sprachen
- Was ist eine Instanz/Referenz//Referenzzähler/GarbageCollection/del
- ~~`https://www.learnpython.org/`~~
- ~~`https://www.programiz.com/python-programming`~~
- KI als Recherche-hilfe benutzen? (KI Nutzung protokollieren -¿ GIT)
- python syntax kontrollstrukturen (if, schleifen, break, pass, try-except) erarbeiten (je ein Beispiel)
- bedingte Ausdrücke `https://openbook.rheinwerk-verlag.de/python/05_001.html` (je ein Beispiel)
- `https://www.datacamp.com/tutorial/python-switch-case` (je ein Beispiel)

KW40

```
#include <stdio.h> // includes "printf" command ;

int add(int varA, int varB ) {
   // inline assembler starts here
   __asm {
      mov eax, varA   ; // move first argument to EAX register
      mov ecx, varB   ; // move second argument to ECX register
      add eax, ecx    ; // EAX = EAX + ECX
   }
}

int main() {
  int result = add(1, 2);     // call C-function "add"
  printf( "1 + 2 = %d\n", result); // output result to console
  return 0;
}
```

Abbildung: Assember und C++ Beispiel

KW40

- Wiederholung
- vorbereitete Beispiele besprechen
- `https://www.programiz.com/dsa/binary-search` wegen Syntax besprechen
- Arraylist in java, listen, echte arrays (array, np.array)
- Zuordnungs Datenstrukturen (dict) `https://openbook.rheinwerk-verlag.de/python/14_001.html`
- Aufgabe Programmiere Lottoziehung als Methode:
  - random.getrand()
  - Algorithmus zum Zufallszahlenziehen muss so programmiert sein, dass keine Zufallszahl zweimal gezogen werden kann.
  - Das heisst, wenn Sie alle 45 Zahlen ziehen müssten, Würden Sie den Zufallszahlengenerator nur 45 mal benutzen dürfen.
  - Ziehe die sechs Zahlen und gib Sie am Bildschirm aus
  - Aufgabe Programmier Lottoziehung Statistik als Methode:
  - Mach 1000, 10000, 100000 usw. Ziehungen
  - Erstelle Dictionary für Statistik, wie oft welche Zahl gezogen wurde
  - Ruf die Statistikmethode nach jede Ziehung auf und inkrementiere den Zähler

KW41

- Wiederholung
- Lotto-Aufgabe besprechen
- Video zu Python type-Hints-checking, data-validation, `https://www.youtube.com/watch?v=fM409bModsE&t=13s` Datentypen (ganze Zahlen (Division?), Gleitkommazahl, Zeichenketten (addieren), Listen, Dictionarys
- divisions `https://www.codingninjas.com/studio/library/floor-division-in-python`
- Variablen sind case sensitiv
- Schlüsselwörter `https://openbook.rheinwerk-verlag.de/python/A_001.html`
- logische Ausdrücke e.g. and / or / lt / eq
- Programmdateien Endung *.py
- Shebang #!/usr/bin/env python3 (MagicLine UNIX) `https://www.delftstack.com/de/howto/python/python-shebang/`
- keine TABS, nur 4 Leerzeichen als Einrückung, Lange Zeilen umbrechen max 80Zeichen (Backslash benutzen)
- Kommentare mit # oder drei Anführungszeichen, pass Anweisung

KW42

- Wiederholung
- Lotto-Aufgabe nochmal ansehen
- list comprehensions
  `https://realpython.com/list-comprehension-python/`
- als HÜ je ein list,set,dict comprehension mit if else vorbereiten
- Auswertungsreihenfolge der Operatoren
- `https://cito.github.io/byte_of_python/read/operator-precedence.html`
- Identität ist Ganzzahl und wie ein Fingerabdruck vergleich mit is
- `https://openbook.rheinwerk-verlag.de/python/07_001.html`
- id(ref1) == id(ref2) oder ref1 is ref2
- Instanzen freigeben mit del
- `https://openbook.rheinwerk-verlag.de/python/07_002.html`
- `https://www.geeksforgeeks.org/garbage-collection-python/`, `https://stackify.com/python-garbage-collection/`

KW43

- TEST 1
- Pokerspielsimulator als Aufgabe über Herbstferien:
  - überlege wie man die Pokerkarten modellieren könnte (vier Farben, 13 Symbole)
  - gib zufällig fünf Karten
  - recherchiere welche Kombinationen beim Pokerspiel exisiteren
  - schreibe Funktionen für die Kombinationen Paar, Drillinge, Poker, Flash, Strasse usw.
  - spiele 100000 mal und zähle die Anzahl der verschiedenen Kombinationen
  - berechne den prozentuellen Anteil einer Kombination zu der Gesamtspieleanzahl
  - recherchiere die richtige Anteile im Netz und vergleiche die Ergebnisse
  - Git!
  - Codingtips beherzigen!!!

KW45

- Pokeraufgabe besprechen
- Referenzen + id + pooling genauer (siehe PDF im GIT)
- mutable/immutable
- `https://openbook.rheinwerk-verlag.de/python/07_003.html`